

Asistente de Emergencias Viales

Verificación, mejoras, pruebas en producción y despliegue · septiembre 2026

Este documento resume todo el trabajo sobre el asistente de voz (LiveKit Agents + RAG sobre el manual de primeros auxilios): la auditoría inicial, los cambios en el agente y en el frontend, los problemas que aparecieron al usarlo en producción (por chat y por voz) y cómo se resolvieron, cómo se verificó cada cosa y qué queda pendiente. Todo está commiteado y el agente está desplegado en LiveKit Cloud.

Resultados en una tabla

Indicador	Al empezar	Ahora
Guion de la Expo (verificaciones automáticas)	18 / 23	23 / 23
«No respira»	«Perdí el acceso al manual», sin RCP	Indica compresiones en el centro del pecho
Hemorragia grave	«Eso no está en mi manual»	«Apretá fuerte sobre la herida...»
«¿Le saco el casco?»	Ignoraba la pregunta	«No le quites el casco; mantené la cabeza y el cuello inmóviles»
RCP en curso («¿y ahora qué?»)	«¿Ves que el pecho se mueve?»	«Seguí comprimiendo sin parar...»
«Sí» al saludo, por voz	«Ya estás geolocalizado...» (derivación falsa)	«¿Hay alguien lastimado?»
Búsquedas en el manual que fallaban	5 de 6 (error 429 de Google)	0 en los temas críticos (caché precalculado)
Respuesta en el chat (producción)	5,6 a 6,8 s	0,4 a 1,8 s
Respuesta por voz (producción, desde que terminás de hablar)	2,0 a 4,3 s	1,3 a 2,9 s
Aviso «Ya estás geolocalizado y la ayuda va en camino»	Dependía del modelo; a veces faltaba o sobraba	Lo agrega el sistema una vez por llamada, solo si se derivó
Tests unitarios	12 pasan	70 pasan

Las mediciones de producción se hicieron con los scripts de prueba desde Argentina, así que incluyen la red. Groq tiene variación propia entre corridas.

1. Diagnóstico inicial

Se corrieron los tests, los escenarios de ensayo.py, la batería test_guion_expo.py y mediciones aisladas de cada fase contra los servicios reales.

Problemas críticos (seguridad)

- **El embedding de Google (plan gratuito) fallaba en ~70 % de las llamadas** con 429, aun espaciadas: el agente no indicaba RCP a alguien que no respira y decía «no está en mi manual» ante una hemorragia.
- **El límite de 7 s del reranker (Cohere Trial) bloqueaba el turno** dentro de una búsqueda con tope de 4 s.
- **El paro no se detectaba después de «inconsciente»**: solo se miraba la primera señal de la llamada.

- **Los avisos de riesgo desaparecían en el turno siguiente:** se agregaban a turn_ctx, que vale solo para ese turno.

Dónde se iba el tiempo

Fase	Medido	Causa
LLM por llamada	~2,5 s	gpt-oss-120b razonaba en modo «medium» (~1900 tokens ocultos)
Llamadas al LLM por turno	2 seguidas	registrar_datos_escena y derivar_a_emergencias casi en cada turno
Embedding (Google)	0,5 a 1,4 s, o error	429 y reintentos
Búsqueda en Supabase	430 a 510 ms	226 ms de ida y vuelta para un corpus de 173 fragmentos
Rerank (Cohere)	0,4 a 1,2 s, o hasta 7 s	Key Trial y throttle bloqueante

2. Cambios para velocidad

Cambio	Detalle
reasoning_effort=«low»	gpt-oss en Groq pasa de ~2,5 s a ~0,6 s por llamada (configurable con LLM_REASONING_EFFORT).
Índice en memoria	Los 173 fragmentos se cargan al arrancar: coseno en numpy y BM25, en menos de 1 ms.
Caché de consultas precalculado	12 consultas canónicas con embedding y ranking rerankeado: los temas críticos salen en 0 ms.
Protocolo adjuntado antes del LLM	Si se detecta un tema, el protocolo va en el mensaje y el modelo responde en una sola pasada.
Menos vueltas al LLM	Sin tool derivar_a_emergencias (la derivación es automática); en turnos críticos, sin herramientas.
Rerank sin espera	Si no hay cupo en la key Trial, se usa el orden del coseno al instante.
Chat sin audio	En la pestaña Test se apaga la voz: antes cada respuesta esperaba ~6 s a que «sonara» para nadie.
Fin de turno por voz	max_delay de 3,0 a 1,2 s: con respuestas cortas («Sí.») el detector esperaba siempre el máximo.

3. Cambios para confiabilidad del manual

- **Orden de búsqueda:** resultado precalculado (0 ms, sin red), después embedding de Google + coseno en memoria + rerank si hay cupo, y si el embedding falla, búsqueda por palabras clave (BM25) en memoria.
- **Textos de respaldo verificados** (protocolos.py): 10 temas (RCP, inconsciente que respira, hemorragia, fuego, atrapado, columna, casco, quemadura, fractura, seguridad de la escena), redactados solo con fragmentos del manual. Si todo lo demás falla, la maniobra igual se dice.
- **Arranque resistente:** el índice se carga en un hilo con tope de espera. Si Supabase está lento (se vio una carga colgada más de 25 s), el agente arranca igual y atiende con los textos de respaldo hasta que el índice esté.

- **Convulsiones no es un tema:** el manual no las trata y la búsqueda devolvía RCP con relevancia 0,27.

4. Cambios para la calidad de las respuestas

- **Detección por turno y por categoría** (paro, atrapamiento, inconsciencia, hemorragia, fuego, convulsión), con derivación automática.
- **Heridos sin depender del modelo:** se deriva con un herido descripto («está tirado en el piso») o confirmado con una respuesta corta a una pregunta sobre heridos («sí, una persona», «una»). «Nadie está lastimado» también se registra solo.
- **Preguntas primero:** si la persona pregunta algo, o vuelve a mencionar un tema ya tratado (aun con errores de tipeo como «le scao el casco»), se responde eso antes que cualquier otra cosa.
- **Una pregunta por vez:** al inicio solo «¿Hay alguien lastimado?»; con un herido, primero si está despierto y después los riesgos.
- **RCP segura:** nunca compresiones sin confirmar que no respira; durante la RCP, «seguí sin parar»; si vuelve a respirar, se deja de comprimir.
- **Resumen verificado + fragmentos:** evita que el modelo tome el primer fragmento al pie de la letra (p. ej. mover a un inconsciente que respira). El resumen del casco ya no incluye la excepción «retíralo si no respira».
- **Corrección del texto antes de hablar** (habla.py): jerga («vía aérea», «fractura»), números («100 a 120», «112»), voseo («verifica», «deja»), tildes en imperativos («Quedáte») y promesas de ayuda cuando no se derivó.
- **Prompt reescrito:** prioridades por turno, nunca pedirle a la persona que llame a nadie, no repetir preguntas ni narrar lo que se registra.

5. Aviso «Ya estás geolocalizado y la ayuda va en camino»

Para la demo la frase es determinística: cuando el sistema deriva al 911 (riesgo de vida, herido descripto o heridos confirmados), se agrega al principio de la primera respuesta, **una sola vez por llamada**. Si el modelo la repite, se quita. Sin derivación no se dice, y tampoco se permite «esperá a que llegue la ayuda». En el panel de debug la derivación aparece como derivar_a_emergencias (automática, con su motivo).

6. Problemas encontrados en producción y cómo se resolvieron

Después del primer despliegue se probó el agente real desde el frontend (chat) y por voz. Cada problema se reprodujo, se corrigió, se agregó como escenario o test y se verificó otra vez en producción.

Síntoma	Causa	Arreglo
Cada respuesta del chat tardaba ~6 s	El agente sintetizaba voz y esperaba a que terminara de «sonar», aunque la pestaña Test no reproduce audio	En el chat de texto se desactiva la salida de audio (también ahorra caracteres de Cartesia)
«está tirado en el piso» no derivaba y el agente decía «llamá ayuda»	La frase no tenía una señal crítica y la derivación dependía de que el modelo llamara a una tool	Detección de herido descripto; el prompt prohíbe pedir que llame
«le scao el casco» se ignoraba	Con el error de tipeo no parecía una pregunta	Si se vuelve a mencionar un tema ya tratado, se trata como pregunta

Síntoma	Causa	Arreglo
«solo retíralo si no respira»	El resumen del casco arrastraba la excepción del manual	Resumen sin la excepción: quitar el casco queda para los médicos
«sí, una persona» no derivaba y volvía a preguntar cuántos heridos	La respuesta no dice «herido» y el modelo no llamó a la tool	Se interpreta la respuesta corta según la última pregunta del agente
Por voz, «Sí» al saludo decía «Ya estás geolocalizado...»	La respuesta generada por adelantado (preemptive) se guardaba como «última pregunta» aunque se descartaba	La última respuesta sale del historial confirmado
Por voz tardaba hasta 4 s en contestar	El detector de fin de turno esperaba 2 s con respuestas cortas	max_delay a 1,2 s

7. Prueba por voz automatizada

prueba_voz.py entra a una sala de LiveKit como si fuera quien llama, «habla» cada frase con audio sintetizado (Cartesia) y muestra lo que entendió el STT, lo que respondió el agente y cuánto tardó en empezar a hablar. Recorre el mismo camino que una llamada real (VAD, fin de turno, Deepgram, generación anticipada, LLM y TTS), así que encuentra fallas que el chat no ve: el bug del «Sí» al saludo solo aparecía por voz.

Vos (voz, producción)	Agente	Tardó
«Sí.»	«¿Hay alguien lastimado?»	1,4 s
«Sí, hay una persona tirada en el piso.»	«Ya estás geolocalizado y la ayuda va en camino. ¿El herido está despierto y responde?»	2,1 s
«¿Le saco el casco?»	«No le quites el casco; mantené su cabeza y cuello inmóviles hasta que llegue la ayuda.»	1,4 s
«...mi acompañante está inconsciente.»	«Ya estás geolocalizado y la ayuda va en camino. Fijate si se le mueve el pecho. ¿Respira?»	2,2 s
«No, no respira.»	«Apoyá el talón de una mano en el centro del pecho y comprimí fuerte y rápido.»	2,0 s
«Ya empecé, ¿y ahora qué hago?»	«Seguí comprimiendo sin parar, dos veces por segundo, hasta que llegue la ayuda...»	2,9 s

Con la variable AGENT_NAME se puede levantar una copia local del agente con otro nombre y probarla por voz sin competir con producción por las llamadas.

8. Cambios en el frontend

- **Panel de debug:** las tarjetas prefetch_protocolo (protocolos adjuntados por el sistema) muestran fragmentos, scores, latencias y el contexto enviado al LLM, con la etiqueta «adjuntado por el sistema · tema» y el campo «Origen» (cache, semantico, lexico, db o respaldo).
- **Pestaña del navegador:** título «Asistente de Emergencias» en lugar de «Create Next App», e ícono de primeros auxilios (cruz blanca sobre verde, ISO 7010) en lugar del triángulo de Vercel. Idioma de la página en español.

9. Archivos del agente

Archivo	Qué hace / qué cambió
agent.py	Carga del índice en segundo plano, contexto_del_turno (triage, heridos, protocolos, preguntas), llm_node con correcciones y aviso 911, chat sin audio, AGENT_NAME
triage.py	Detección por categoría, herido descrito y confirmado, derivación automática, orden de preguntas
prompts.py	Prompt reescrito
protocolos.py (nuevo)	Temas críticos, detección y textos de respaldo
habla.py (nuevo)	Correcciones del texto hablado y aviso 911
rag/index.py, rag/cache_consultas.json (nuevos)	Índice en memoria (coseno + BM25) y caché precalculado
rag/retriever.py, store.py, embeddings.py, rerank.py, ratelimit.py	Orden de búsqueda nuevo, reintentos y rerank sin espera
ingestion/cache_consultas.py (nuevo)	Regenera el caché después de cada reingesta
prueba_voz.py (nuevo)	Prueba de punta a punta por voz
ensayo.py, test_guion_expo.py	Mismo contexto del turno que el agente; escenarios reales de producción
tests/test_turno.py, tests/test_agente.py (nuevos)	Lógica determinística, aviso 911, heridos, índice no cargado, bug del «Sí»
requirements.txt	livekit-agents 1.6.8 (la versión probada) y numpy explícito

10. Cómo se verificó

- **Tests unitarios:** 70 pasan (9 se saltean porque requieren el corpus reconstruido).
- **Guion de la Expo** contra Groq real: 23/23.
- **Escenarios de ensayo.py**, incluidas las conversaciones reales de producción (moto_produccion, herido_confirmado).
- **Chat contra producción:** script que se conecta como la pestaña Test y mide cada respuesta.
- **Voz contra producción:** prueba_voz.py, con la conversación del bug y el camino crítico de RCP.
- **Frontend:** TypeScript, ESLint y next build sin errores.

11. Estado del código y despliegue

Repo	Rama	Commits	Estado
asistente_de_emergencias (agente)	dev	27d68a8, 58a42c8, 4ec52c6, dc42a33	Desplegado en LiveKit Cloud (us-east), versión 7JHsPeoueJgv
asistente-frontend	main	fa9830a, f45aaa7, b3a2e58	En main; Vercel despliega desde ahí

12. Cómo probarlo

Por texto, en la consola:

```
./venv/Scripts/python.exe ensayo.py --interactivo
./venv/Scripts/python.exe ensayo.py --escenario herido_confirmado
./venv/Scripts/python.exe test_guion_expo.py
```

Por voz, contra producción o contra una copia local:

```
./venv/Scripts/python.exe prueba_voz.py "Sí." "Choqué con una moto." "¿Le saco el casco?"

$env:AGENT_NAME="asistente-emergencias-prueba"; ./venv/Scripts/python.exe agent.py dev
./venv/Scripts/python.exe prueba_voz.py --agente asistente-emergencias-prueba
```

Después de cada reingesta de PDFs y para desplegar:

```
./venv/Scripts/python.exe -m ingestion.cache_consultas --mostrar
lk agent deploy
```

13. Pendientes y riesgos

- **El agente se despliega desde dev:** la rama main del agente no tiene estos cambios.
- **Facturación en Google AI Studio** (opcional, centavos): eliminaría los 429 en consultas fuera del caché, que hoy caen a la búsqueda por palabras.
- **Fin de turno:** el detector da probabilidades bajas en español (0,3 a 0,5) y casi siempre espera el máximo (1,2 s). Bajar su umbral para español ahorraría ~0,7 s más por turno, con riesgo de cortar a quien hace pausas.
- **Detalle menor:** si la persona no contesta la pregunta, el agente la repite (p. ej. «¿Hay alguien lastimado?» dos veces).
- **Versiones:** el Dockerfile usa Python 3.11 y el venv local 3.13.
- **Métricas:** tres scripts de metrics/ importan search_similarity, que no existe desde antes de estos cambios.
- **README:** dice que los embeddings son de Cohere; son de Gemini.
- **GitHub:** quedó la rama feature/debug-prefetch-protocolos del frontend, ya integrada en main.